LLM Inference Curriculum

DENNIS KENNETZ

FEB 14, 2026

One of the biggest things I see people struggle with early is this:

- you need to get hands on and build
- know that what you build won't matter, it's for your learning
- you'll throw it away and move on

There's no substitute for applying the theory if you want to be an engineer. Theory will only take you so far if you want to work for a product company. You have to be able to apply it, and applying it looks like building something.

Here's a curriculum I'd follow for learning LLM inference today. The goal is simple: build small, learn the mechanics, then scale the problem until the bottlenecks change.

1) CPU-based model serving (start here)

Start at the trained model and work forward:

- model architecture + layers (what's actually in the "thing" you're serving)
- tokenization (how "text" becomes IDs)
- forward pass (and just enough backward pass context to understand compute vs memory)

Then zoom into the serving mechanics:

Model lives as files on disk → pulled into CPU memory → server process loads weights → you respond to requests.

From there, the important questions are practical:

- Quantization on CPU: int8 vs int4 vs int2 — how it changes memory footprint, latency, and output quality
- Storage matters: HDD vs SSD vs NVMe — fast storage means weights load faster (and this matters if you restart, autoscale, deploy, or recover)
- The plumbing: disk → RAM, and where PCIe fits into the story
- The request path: tokenization in, logits out, decoding out — what happens on the way in and the way out

Practical 1: build a simple CPU inference server on your laptop which can receive and respond to requests. Keep it intentionally basic. Your job is to understand the end-to-end path, not to ship it.

2) GPU-based model serving (everything changes)

Now ask: why does a GPU help so much for inference?

- GPU architecture vs CPU: cores, tensor cores, shared memory, registers, global memory
- what “parallel” really means in practice (and what isn’t parallel)

Then the big shift: the programming model.

You’re now writing CPU code and GPU code (even if frameworks hide it). Learn the layers:

- low level: CUDA / ROCm
- higher level: PyTorch / Triton

Then tie it back to serving mechanics:

- disk → CPU RAM → GPU VRAM (what has to happen, and why it's not free)
- GPU memory is usually much smaller than CPU memory — constraints show up fast

Now you earn the real inference concepts:

- latency vs throughput: GPUs are high throughput by design; how do inference engines fight latency and maximize utilization?
- model size math: parameter count × precision → weight memory, then add activation/KV overhead
- prompt length: revisit the forward pass and see why "supports 1,000,000 tokens" is a big deal (it's a memory + bandwidth story, not marketing)
- CPU offloading: what it is, when it's necessary, and what it costs
- KV-cache: what it stores, why it exists, why it dominates memory, and why it's the source of so many "why is my server slow/oom?" moments

Make yourself do the math: how much memory should weights take? how much for KV-cache per token? what's your max context at a given precision?

Practical 2: port your CPU server to GPU, or build a minimal GPU inference server. Time it. Measure throughput vs latency. Watch what breaks.

3) Multi-GPU model serving (the network becomes the product)

Once you understand one GPU, the naive question is: "can I just use more?"

Yes… and now the bottleneck shifts.

You introduce interconnect and bandwidth:

- GPUs on the same node (NVLink / AMD's intra-node fabrics)
- GPUs connected primarily via PCIe (why that becomes the bottleneck fast)

Then you learn the inference parallelism toolbox:

- data parallel
- tensor parallel
- pipeline parallel
- expert parallel

And you stop treating "distributed" as magic and start treating it as communication + scheduling.

- collective communications: what they are and why they matter
- NCCL / RCCL
- all-reduce, all-gather, all-to-all — and the trade-offs you pay in real systems

Also worth understanding at this stage:

- offloading (CPU + storage) when VRAM is the limiting factor
- high performance storage paths (and why "fast storage" comes back as a recurring theme)
- "offline" KV strategies and what they unlock (and what they complicate)

Practical 3 (choose one):

- break down an inference engine (vLLM, NVIDIA TensorRT, SGLang) from "prompt in" to "tokens out"

- or write a single-node, multi-GPU model server and learn where the scaling stops being linear

4) Multi-node serving + data center design (where it gets fun)

Once you leave a single node, inference becomes a systems problem: networking, storage, placement, and architecture.

At this stage you're learning distributed HPC fundamentals through the lens of inference:

Networking

- RDMA: what it is, what matters, why people obsess over it
- network topology: what different topologies do to latency, bisection bandwidth, and tail behavior
- are some topologies "better" for inference than training? (yes, depending on workload shape)

Storage

- parallel filesystems
- distributed KV-cache ideas
- what storage enables when compute and memory are constrained

Compute proximity

- speed of light is a feature and a limitation
- hops matter
- topology matters
- placement is an optimization problem

Load balancing

Inference load balancing is different from "normal" distributed compute:

- requests are bursty
- context is sticky
- batching and KV make the system stateful in weird ways
- tail latency is the real enemy

Practical 4 (choose one):

- break down a multi-node inference framework like llm-d or NVIDIA Dynamo
- or go deep on one pillar (distributed storage, networking, scheduling/placement) and map it back to real inference bottlenecks

If you do this in order, you'll notice something important: every stage introduces a new limiting factor.

CPU: disk + memory + basic serving path

GPU: VRAM + kernel efficiency + batching

multi-GPU: communication + collectives + scheduling

multi-node: topology + RDMA + placement + tail latency + storage

That's the mindset shift: you're not "learning LLM inference" — you you're learning how bottlenecks move as you scale the same problem.

Build small, throw it away, build the next thing. That's the curriculum.